Exploratory Data Analysis (EDA)

session features extracted from vsftpd logs (17 features per session)

```
import pandas as pd
from ftp_ids.parsers.vsftpd_parser import VsftpdParser
from ftp_ids.core.session_builder import build_sessions
from ftp_ids.core.feature_extractor import FeatureExtractor

parser = VsftpdParser()
fe = FeatureExtractor()

with open("../data/samples/vsftpd.log") as f:
    events = [parser.parse_line(line) for line in f if line.strip()]

sessions = build_sessions([e for e in events if e])

df = pd.DataFrame([
    {
        "src_ip": s["src_ip"],
        "user": s["user"] or "none",
        "start_time": s["start_time"],
        "end_type": s["end_type"],
        **fe.extract(s),
    }
    for s in sessions
])
df.head()
```

	src_ip	user	start_time	end_type	total_events	failed_logins	downloads	uploads	unique_commands	unique_files	...	total_bytes	avg_speed
0	178.65.243.243	deploy_user	2026-04-16 05:44:27	clean	124	0	1	10	15	8	...	3444037.0	534.941818
1	170.142.236.64	deploy_user	2026-04-16 21:46:07	clean	136	0	1	12	15	10	...	3487511.0	532.996923
2	132.125.189.73	deploy_user	2026-04-16 23:56:46	clean	148	0	1	14	15	12	...	3500263.0	947.926000
3	115.235.179.79	deploy_user	2026-04-17 00:05:13	clean	118	0	1	9	15	7	...	3520853.0	561.308000
4	221.89.154.104	deploy_user	2026-04-17 10:25:17	clean	142	0	1	13	15	11	...	3535249.0	793.707143

5 rows × 21 columns

1. Overview

```
print("sessions:", len(df))
print("unique IPs:", df["src_ip"].nunique())
print("unique users:", df["user"].nunique())
print("date range:", df["start_time"].min(), "to", df["start_time"].max())

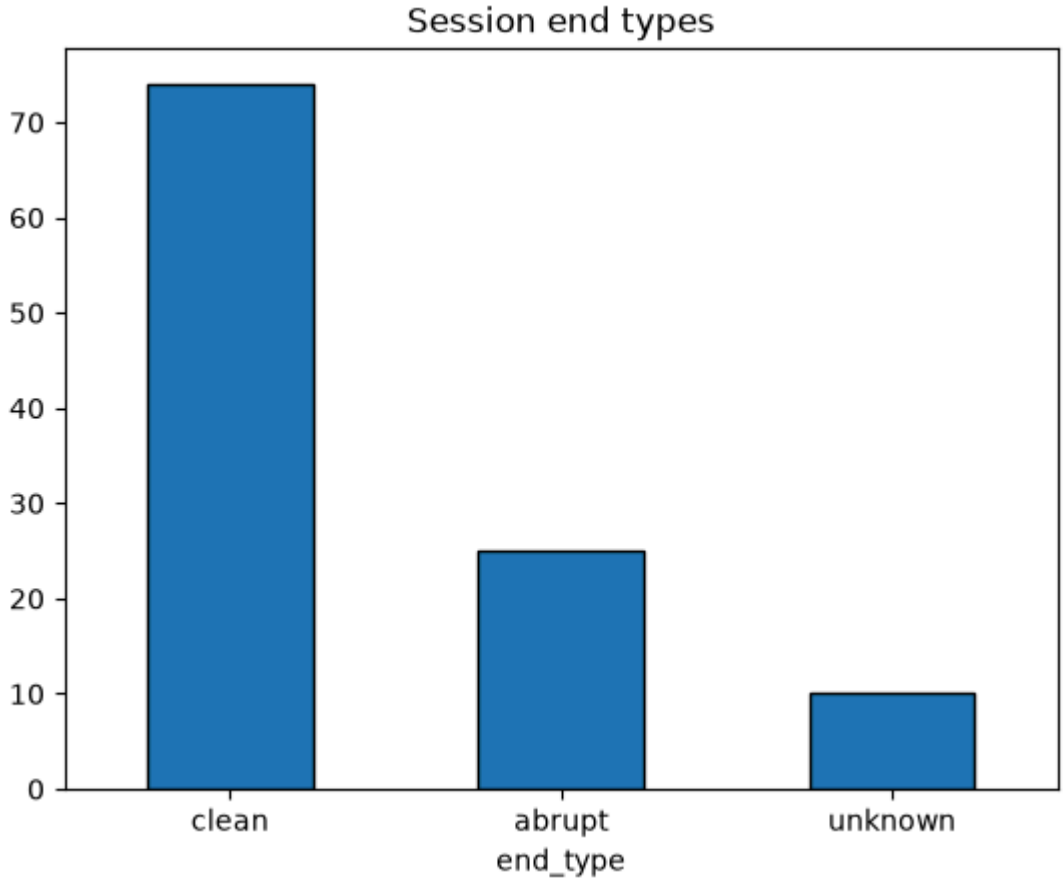
df["user"].value_counts()
```

```
sessions: 109
unique IPs: 86
unique users: 5
date range: 2026-04-16 05:44:27 to 2026-06-25 06:43:00
```

```
user
deploy_user    98
none            4
anonymous       4
admin           2
fakeuser        1
Name: count, dtype: int64
```

```
import matplotlib.pyplot as plt

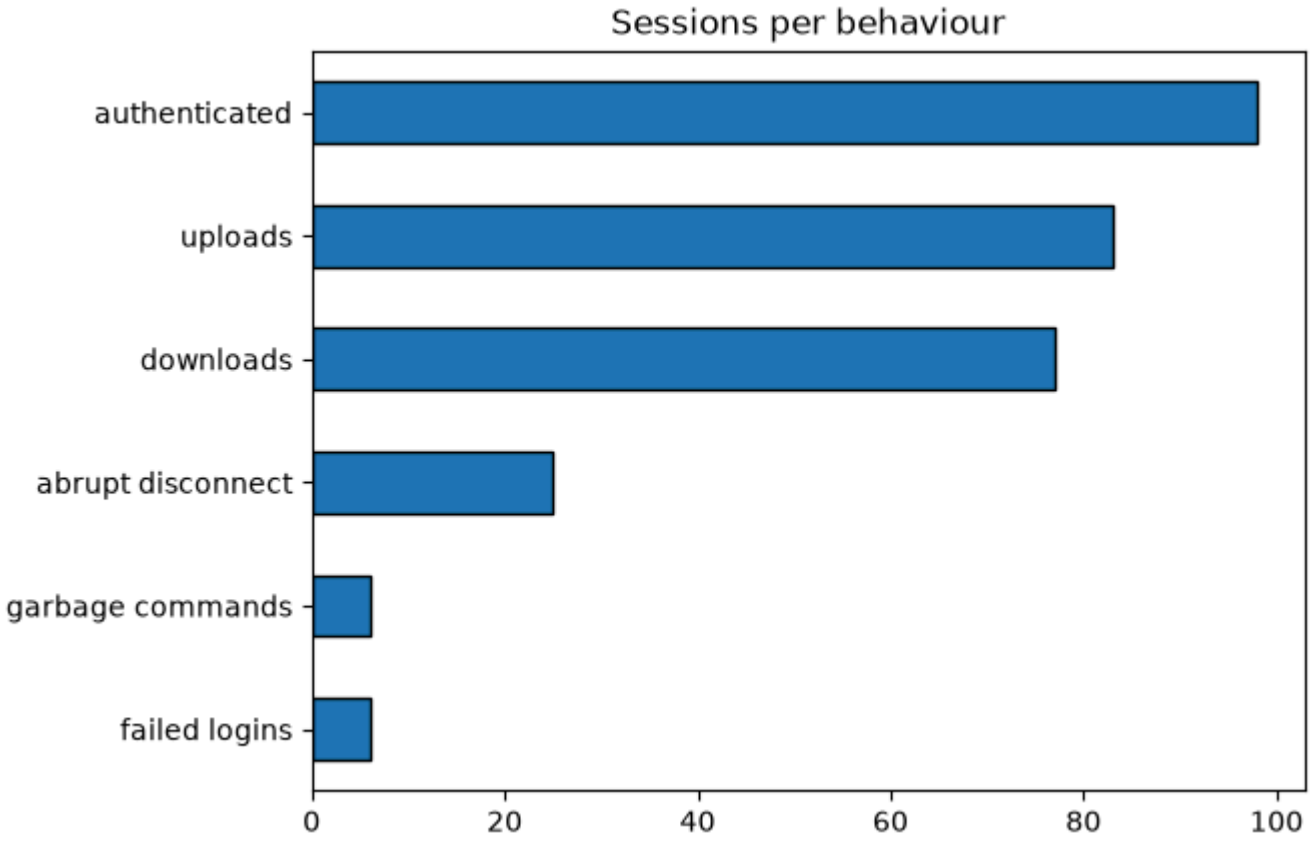
df["end_type"].value_counts().plot(kind="bar", rot=0, edgecolor="black", title="Session end
plt.show()
```



2. Behaviours

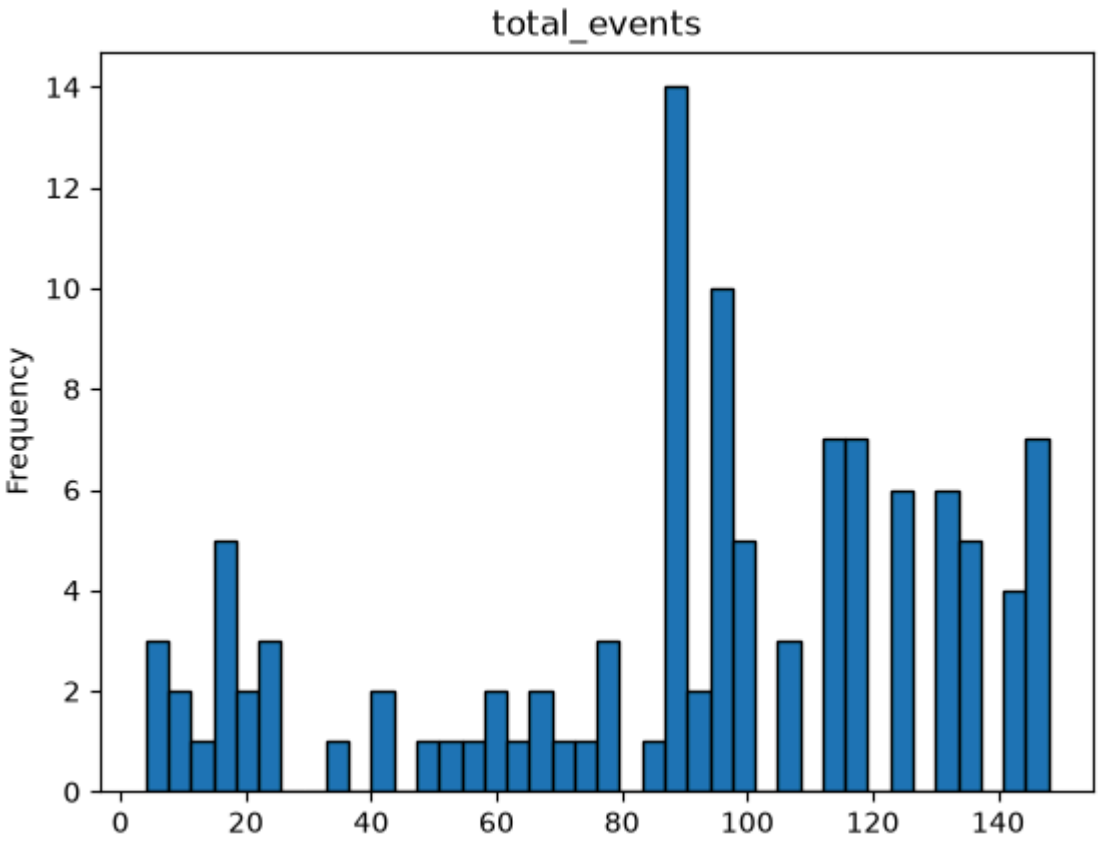
```
behaviours = pd.Series({
    "authenticated": df["is_auth"].sum(),
    "failed logins": (df["failed_logins"] > 0).sum(),
    "garbage commands": (df["garbage_cmd_ratio"] > 0).sum(),
    "abrupt disconnect": df["abrupt_disconnect"].sum(),
    "uploads": (df["uploads"] > 0).sum(),
    "downloads": (df["downloads"] > 0).sum(),
})

behaviours.sort_values().plot(kind="barh", edgecolor="black", title="Sessions per behaviour
plt.show()
```



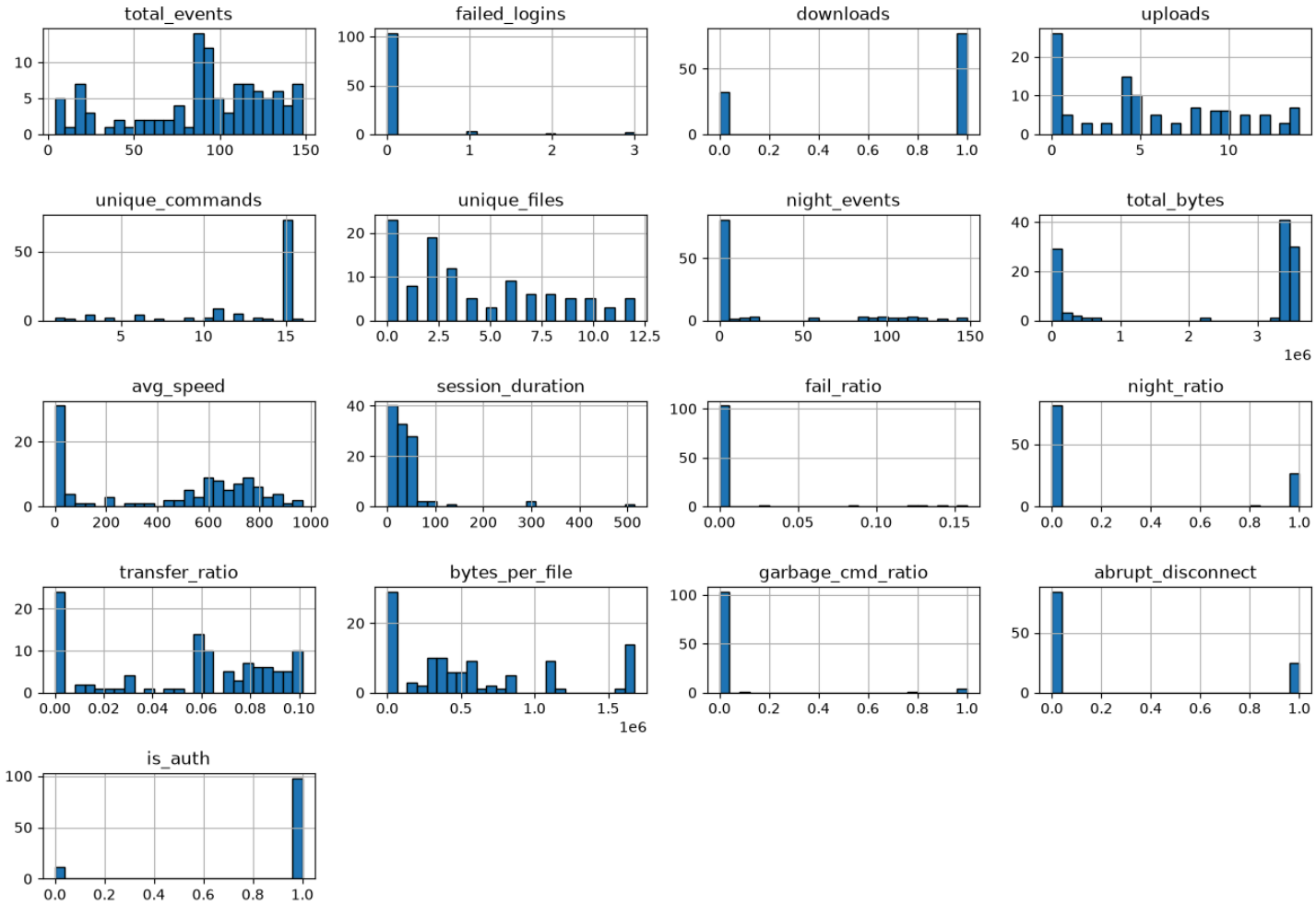
3. Feature distributions

```
df["total_events"].plot(kind="hist", bins=40, edgecolor="black", title="total_events")
plt.show()
```




```
from ftp_ids.core.feature_extractor import FEATURE_NAMES
```

```
df[FEATURE_NAMES].hist(bins=25, figsize=(13, 9), edgecolor="black")
plt.tight_layout()
plt.show()
```

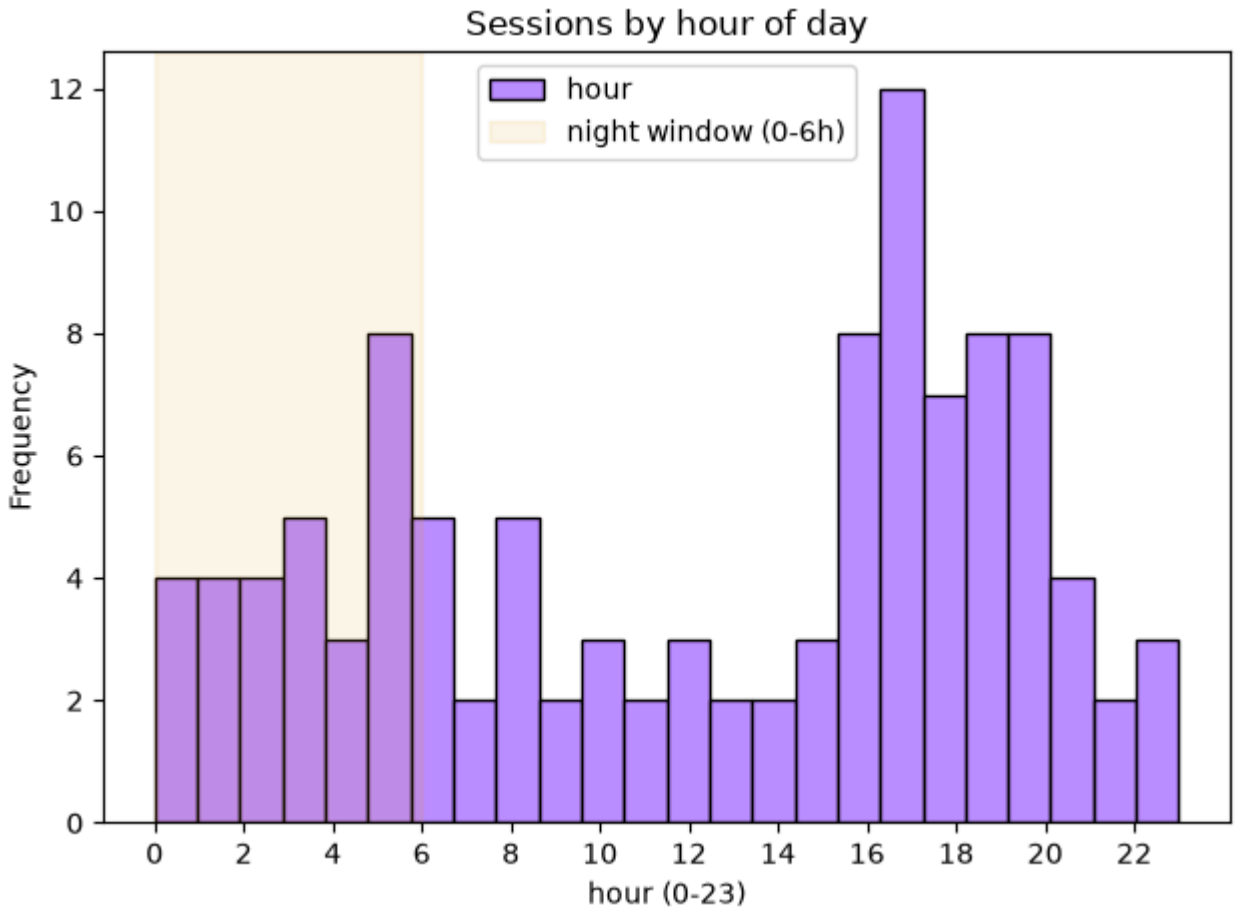


4. Temporal pattern

```
df["hour"] = df["start_time"].dt.hour

ax = df["hour"].plot(kind="hist", bins=24, color="#bc8cff", edgecolor="black")
ax.set_title("Sessions by hour of day")
ax.set_xlabel("hour (0-23)")
ax.set_xticks(range(0, 24, 2))
ax.axvspan(0, 6, color="#f59e0b", alpha=0.1, label="night window (0-6h)")
ax.legend()
plt.tight_layout()
plt.show()

night = (df["hour"] < 6).sum()
print(f"Night sessions (00:00-06:00): {night}/{len(df)} ({100*night/len(df):.0f}%)")
```

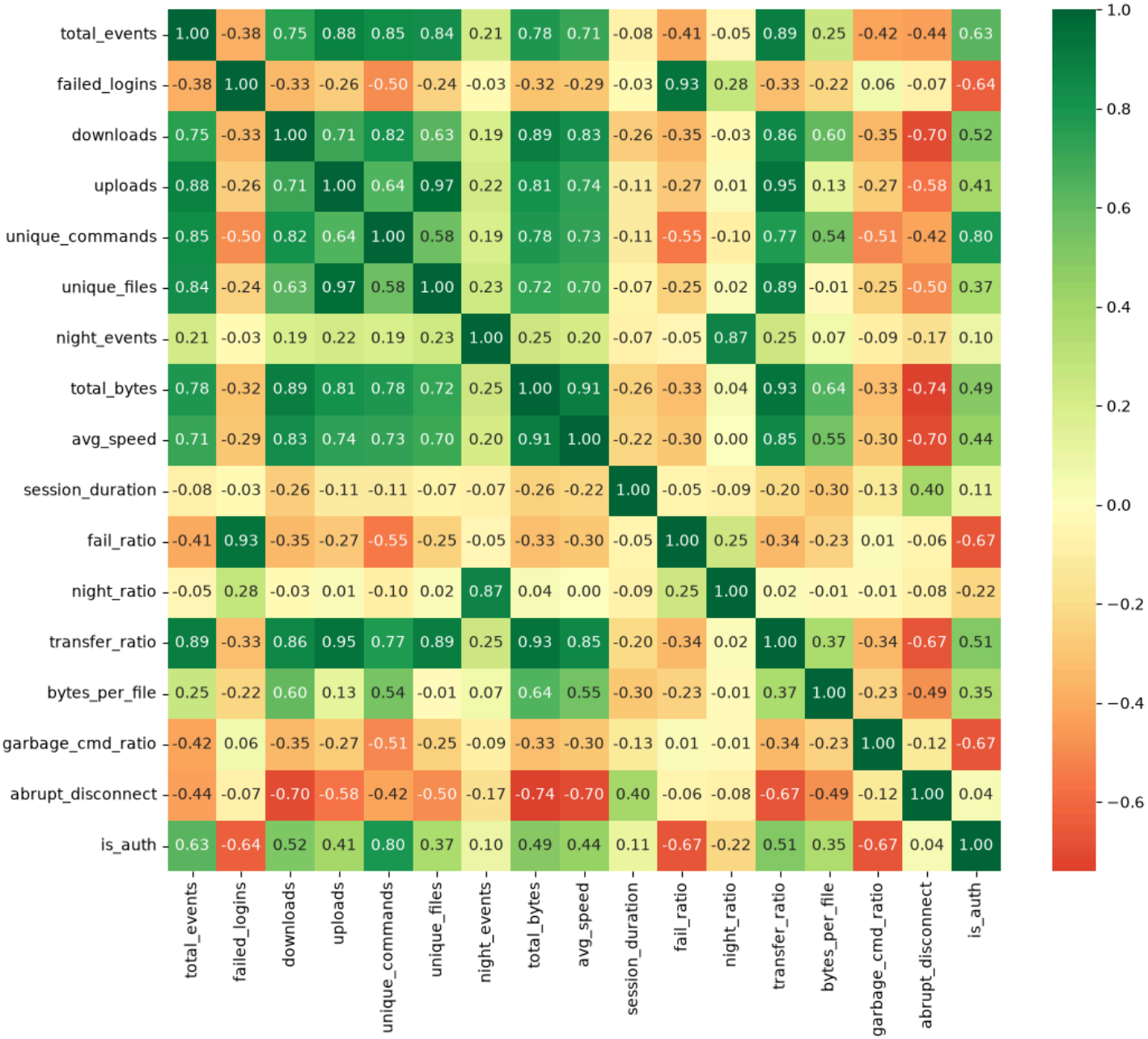



```
Night sessions (00:00-06:00): 28/109 (26%)
```

5. Feature correlation matrix

```python
import seaborn as sns

plt.figure(figsize=(12, 10))
sns.heatmap(df[FEATURE_NAMES].corr(), annot=True, fmt=".2f", cmap="RdYlGn", center=0)
plt.show()
```



6. Class balance (heuristic preview)

a session is probably normal if it logged in as deploy_user, otherwise it is probably hostile

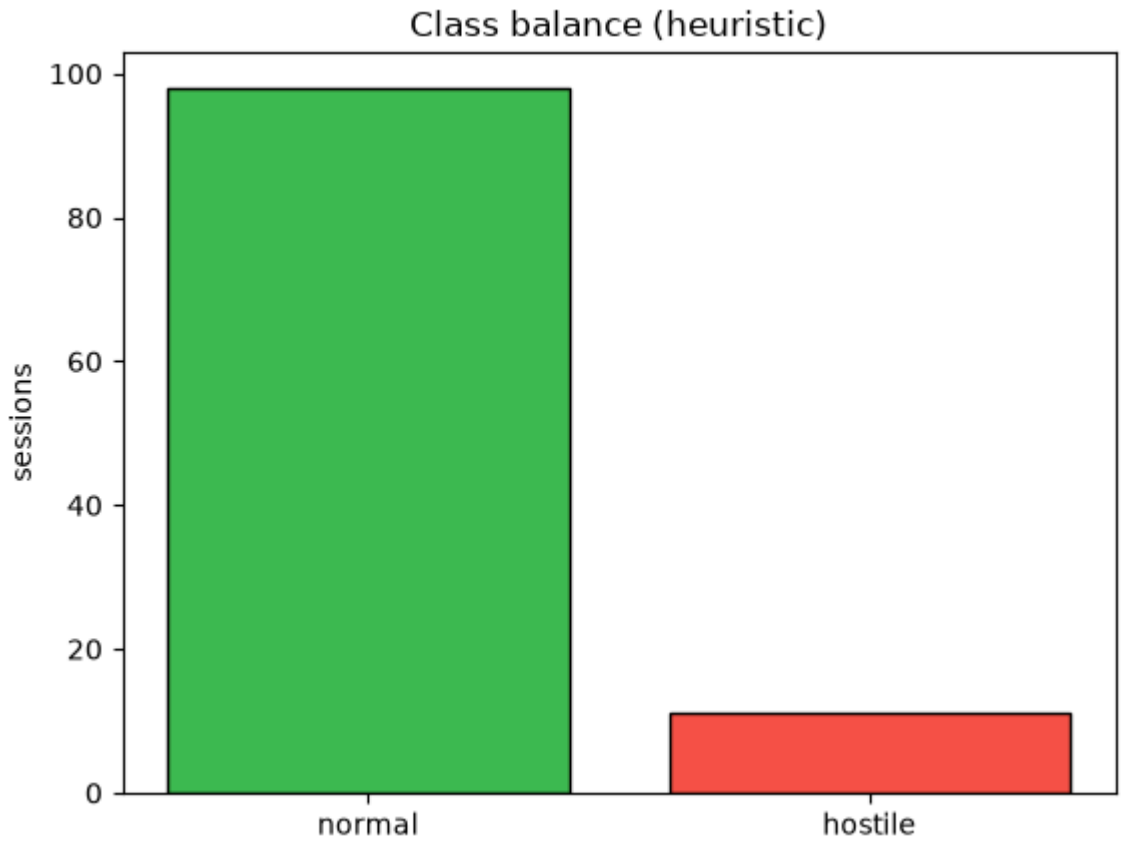
```
df["label"] = "hostile"
df.loc[(df["user"] == "deploy_user") & (df["is_auth"] == 1), "label"] = "normal"

n_normal = (df["label"] == "normal").sum()
n_hostile = len(df) - n_normal

print(f"normal:  {n_normal} ({100 * n_normal / len(df):.0f}%)")
print(f"hostile: {n_hostile} ({100 * n_hostile / len(df):.0f}%)")

plt.bar(["normal", "hostile"], [n_normal, n_hostile], color=["#3fb950", "#f85149"], edgecol
plt.title("Class balance (heuristic)")
plt.ylabel("sessions")
plt.show()
```

```
normal:  98 (90%)
hostile: 11 (10%)
```

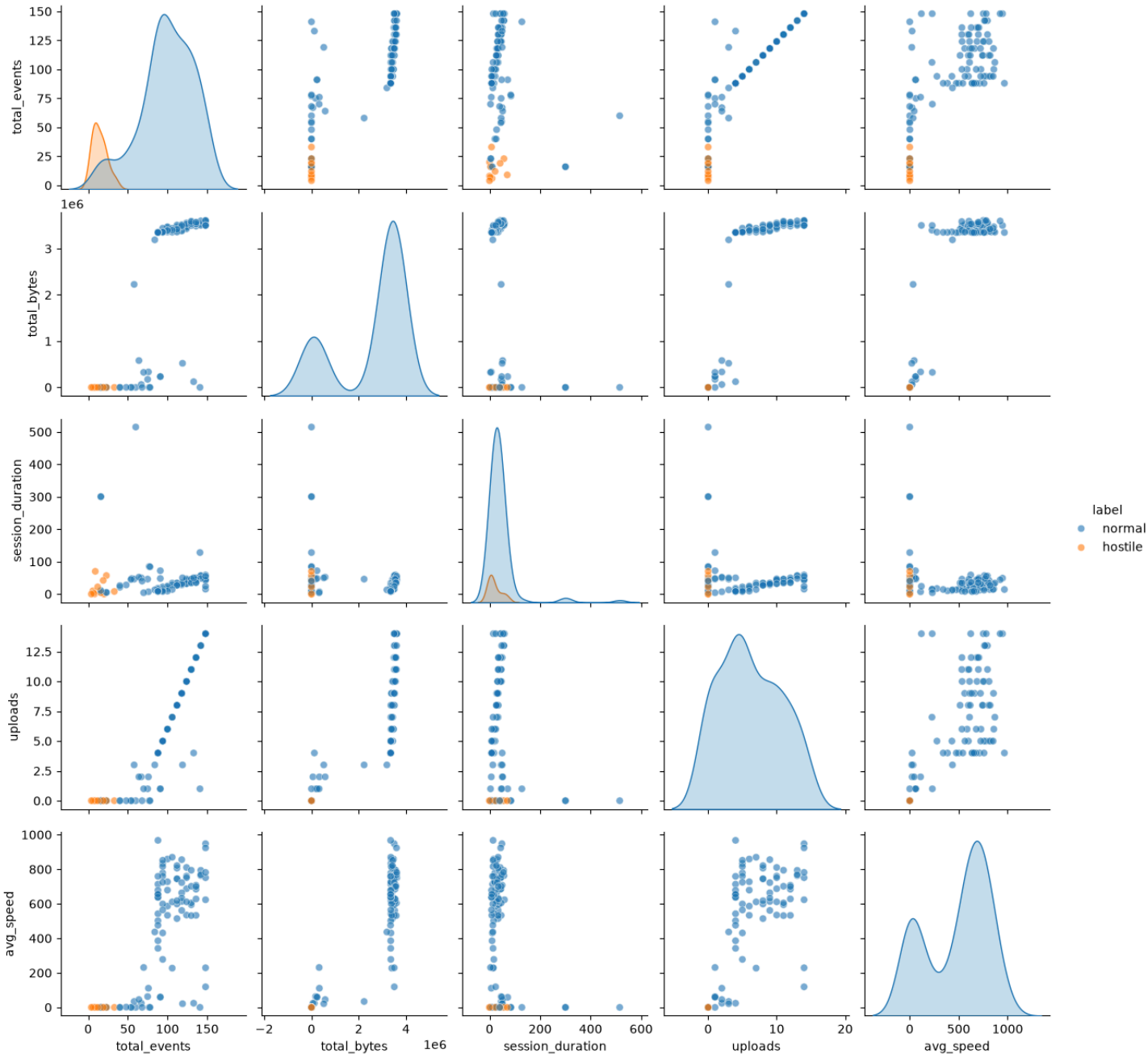


7. Deploy session profile

this pairplot compares 5 features against each other, as our dataset mainly consists of legitimate CI/CD deployments, we expect a consistent cluster which confirms that deploys follow a predictable pattern that our anomaly detection algorithm (Isolation Forest) can learn as normal behavior

orange dots are hostile sessions

```
sns.pairplot(
    df[["total_events", "total_bytes", "session_duration", "uploads", "avg_speed", "label"]
    hue="label",
    plot_kws={"alpha": 0.6},
)
plt.show()
```



8. Summary

```
print("sessions:", len(df), "from", df["src_ip"].nunique(), "IPs")
print("date range:", df["start_time"].min().date(), "to", df["start_time"].max().date())
print("median events:", df["total_events"].median())
print("authenticated:", df["is_auth"].sum())
print("garbage commands:", (df["garbage_cmd_ratio"] > 0).sum())
print("abrupt disconnects:", df["abrupt_disconnect"].sum())
print("failed logins:", (df["failed_logins"] > 0).sum())
print("night sessions:", night)
print("normal:", n_normal, "hostile:", n_hostile)
```

```
sessions: 109 from 86 IPs
date range: 2026-04-16 to 2026-06-25
median events: 94.0
authenticated: 98
garbage commands: 6
abrupt disconnects: 25
failed logins: 6
night sessions: 28
normal: 98 hostile: 11
```

this section summarizes the previous pairplot for normal sessions only, using just the relevant features for a deployment session profile

```
deploys = df[df["label"] == "normal"]

print("clean exits:", (deploys["end_type"] == "clean").sum(), "/", len(deploys))
deploys[["uploads", "total_bytes", "avg_speed", "session_duration"]].agg(["min", "mean", "m
```

```
clean exits: 73 / 98
```

	uploads	total_bytes	avg_speed	session_duration
min	0.0	0.0	0.0	4.0
mean	6.2	2577260.5	486.4	41.8
max	14.0	3611943.0	967.2	515.0